Technical Documentation: AI Travel Planning Assistant

Agentic AI capstone project using LangChain, Groq, FastAPI, and a custom frontend.

1. Project Overview

The system plans a trip from a natural language or form request. An LLM agent decides which tools to call (flights, hotels, places, weather, and budget), collects real data from local datasets and a free weather API, and returns a structured plan. Local tabs (Hotels, Places, Itinerary) work without the LLM, so they are fast and free.

2. System Architecture

```
Browser (HTML, CSS, JavaScript)
   |
   |  fetch() calls
   v
FastAPI backend (api.py)
   |-- /api/plan, /api/plan/json  --> Agent (agent/travel_agent.py)
   |                                      |-- Groq LLM (openai/gpt-oss-20b)
   |                                      |-- Tools (tools/*.py)
   |                                            |-- flights.json
   |                                            |-- hotels.json
   |                                            |-- places.json
   |                                            |-- Open-Meteo API
   |-- /api/hotels, /api/places, /api/itinerary  --> local data only
   |-- /api/chat  --> Groq LLM (no tools)
   v
Browser localStorage (favorites, saved trips)
```

3. Components

Layer	File(s)	Responsibility
Backend API	api.py	Routes, request validation (Pydantic), error handling
Agent	agent/travel_agent.py	Groq LLM, tool-calling loop, system prompt, JSON mode
Query builder	agent/query_builder.py	Turns form values into a natural language query
Tools	tools/*.py	Flight, hotel, places, weather, and budget functions
Data	data/*.json	Sample flights, hotels, and places datasets
Frontend	frontend/index.html, style.css, script.js	Pages, forms, results, and local tabs
Storage	frontend/storage.js	Favorites and saved trips in localStorage
Chat	frontend/chat.js	Chat panel that calls /api/chat
Deployment	requirements.txt, runtime.txt	Python packages and version for Render

4. Agent Tools

Tool	Input	Output	Data source
search_flights	source, destination, travel date	Matching flights, cheapest and fastest	flights.json
search_hotels	city, min stars, max price	Ranked hotels, top recommendation	hotels.json
search_places	city, type, min rating	Ranked places, top 3	places.json

get_weather_forecast	city, start date, end date	Daily max/min temperature, rain %, condition	Open-Meteo A
estimate_trip_budget	flight price, hotel price, days, daily expense	Flight, hotel, local, and total cost	Calculation

Each tool returns plain dictionaries. The agent receives them as JSON, and list outputs are shortened to 5 items to save tokens.

5. Agent Flow

1. The user submits the form (cities, dates, travellers, budget, preferences).
2. The query builder converts the form values into a natural language query.
3. The agent sends the system prompt and query to the Groq LLM, with the enabled tools attached.
4. If the LLM asks for tools, the agent runs them, adds the results as tool messages, and calls the LLM again.
5. This loop runs for up to 12 rounds. When the LLM answers without tool calls, the answer is returned.
6. For /api/plan/json, the final answer is parsed as JSON. If parsing fails, the agent retries once.

6. Prompt Design

The system prompt is built in build_system_prompt(). It is generated per request, with today's date and only the enabled tools. It contains two rule groups:

Tool usage rules: call each tool once, stop when data is enough, use travel_date, pick cheapest or fastest flight by preference, and call search_places only once.

Answer rules: use only tool data, no invented places or prices, correct travellers and room math, budget comparison, one-way flight note, amenity filter, closest alternatives when no exact match, and a fallback message when interests have too few places.

For JSON output, a fixed JSON schema is appended to the prompt, and the answer is validated with json.loads().

7. Key Design Decisions

- Local tools for hotels, places, and itinerary tabs, so the Groq limit is not used for browsing.
- Shortened tool outputs (5 items per list) to reduce tokens per request.
- Temperature set to 0.2 for consistent answers.
- Budget math done in a tool, not by the LLM, to avoid calculation errors.
- Browser storage for favorites and saved trips, so no database is needed for the demo.
- Fallback messages (closest hotels, non-matching interests) instead of empty results.

8. API Endpoints

Method	Endpoint	Uses Groq	Purpose
GET	/api/health	No	Health check
POST	/api/plan	Yes	Agent trip plan (Markdown text)
POST	/api/plan/json	Yes	Agent trip plan (structured JSON)
POST	/api/chat	Yes	Travel assistant chat (no tools)
GET	/api/cities	No	Cities list from hotels.json

GET	/api/hotels	No	Filtered hotel search
GET	/api/place-options	No	Cities and types for filters
GET	/api/places	No	Filtered place search
GET	/api/itinerary	No	Local day-wise itinerary and budget

9. Deployment

- Code is hosted on GitHub. Every push to main triggers a redeploy on Render (free web service).
- Start command: uvicorn api:app --host 0.0.0.0 --port $PORT
- Python version pinned in runtime.txt. Packages listed in requirements.txt.
- GROQ_API_KEY is stored as a Render environment variable, never in the repository.
- The free plan sleeps after inactivity, so the first request can take 30 to 60 seconds.

10. Limitations

- Sample datasets, not live prices or availability.
- Flight dates from 2025 are shown on the requested date using the dataset schedule (documented in the response).
- Weather forecasts are reliable for about 16 days only.
- Groq free tier has token-per-minute and token-per-day limits, which restrict heavy testing.
- Browser storage is per browser; favorites and trips do not sync across devices.

11. Future Work

- Replace sample data with live APIs or scraped data.
- Add user accounts and a database (SQLite or PostgreSQL) for trips and favorites.
- Add automated tests (pytest) and code style checks (flake8).
- Add LLM evaluation tracking with MLflow or LangSmith.